

Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Laboratory Work 7:

Three Small Programs: Processes, Redirection, a Pipe

Elaborated: st. gr. FAF-243

Șoimu Ionuț

Verified: asist. univ.

Reițman Patricia

Chișinău – 2026

Contents

1 Lab 7: Track A – Three Small Programs: Processes, Redirection, a Pipe **2**
1.1 Before you start . . . 2
1.2 Part 1. fork + exec + wait . . . 2
1.2.1 Program Implementation: run.c . . . 2
1.2.2 Compilation and Baseline Execution . . . 3
1.2.3 Child Replacement Tests . . . 3
1.3 Part 2. A zombie, on purpose . . . 4
1.3.1 Constructing a Zombie Process: zombie.c . . . 4
1.3.2 Process Table Observation . . . 5
1.4 Part 3. Redirection: ls -l ¿ out.txt . . . 6
1.4.1 Descriptor Table Manipulation: redirect.c . . . 6
1.4.2 Execution and Output Verification . . . 6
1.5 Part 4. A pipe: ls /etc — wc -l . . . 7
1.5.1 Inter-Process Communication: pipeline.c . . . 7
1.5.2 Comparison with Standard Shell Pipeline . . . 8
1.5.3 Intentional Bug Experiment: The Hanging Reader . . . 8
1.6 Optional Extension: Three-Stage Pipeline (pipeline3.c) . . . 9
1.6.1 Implementation: pipeline3.c . . . 9
1.6.2 Verification . . . 10

2 Conclusion **10**

1 Lab 7: Track A – Three Small Programs: Processes, Redirection, a Pipe

1.1 Before you start

Purpose and Scope

In operating systems theory, three fundamental questions arise regarding execution and inter-process communication:

(a) How are processes spawned and transformed into new programs?
(b) How does the operating system decouple user programs from specific output sinks (`stdout` redirection)?
(c) How do independent, isolated processes stream data directly to one another via pipes?

The objective of this laboratory work is to investigate these mechanisms on Linux through hands-on system programming in C. We write and dissect four foundational programs:

- `run.c`: Demonstrates process creation and replacement via `fork()`, `exec()`, and `waitpid()`.
- `zombie.c`: Deliberately constructs and inspects a zombie (defunct) process to understand process table lifecycle mechanics.
- `redirect.c`: Reconstructs shell-style file redirection (`ls -l > out.txt`) using descriptor table manipulation.
- `pipeline.c`: Constructs an inter-process communication pipeline (`ls /etc | wc -l`) connecting two child processes via an anonymous kernel buffer.
- `pipeline3.c` (extension): Extends the pipeline model into a three-stage pipeline (`ls /usr/bin | grep x | wc -l`) using dual pipes and three child processes.

Environment Setup and Compiler Identification

```
➜  ~ mkdir -p ~/os-lab7 && cd ~/os-lab7
➜  os-lab7 whoami
ionut
➜  os-lab7 gcc --version | head -1
gcc (GCC) 16.2.1 20260810
```

1.2 Part 1. fork + exec + wait

1.2.1 Program Implementation: run.c

This program models the three essential steps executed by a command shell when running an external command:

Step 1: **Fork:** The calling parent creates an exact copy of its address space via `fork()`.
Step 2: **Exec:** The newly created child replaces its address space, text, and data with the target binary using `execlp()`.
Step 3: **Wait:** The parent suspends execution via `waitpid()`, reaping the child's termination status code upon completion.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c

/* the three steps of the shell for the command:
   ls -l
*/
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

1.2.2 Compilation and Baseline Execution

```
➜  os-lab7 gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12725
child:  my PID is 12732, my parent is 12725
total 24
-rwxr-xr-x 1 ionut ionut 16472 Oct 19 15:39 run
-rw-r--r-- 1 ionut ionut   910 Oct 19 15:39 run.c
parent: child 12732 ended with status 0
```

1.2.3 Child Replacement Tests

We modify line 19 of `run.c` to execute alternative binaries:

(a) **Executing `date`:**

```
# Replaced with: execlp("date", "date", (char *)NULL);
➜  os-lab7 gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12735
child:  my PID is 12742, my parent is 12735
Mon Oct 19 03:39:24 PM EEST 2026
parent: child 12742 ended with status 0
```

(b) **Executing Non-Existent Program `nosuchprog`:**

```
# Replaced with: execlp("nosuchprog", "nosuchprog", (char *)NULL);
➜  os-lab7 gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12745
child:  my PID is 12752, my parent is 12745
```

```
exec: No such file or directory
parent: child 12752 ended with status 127
```

Observe:

1. **The line perror(”exec”) is in the child, right after execlp. Why is it printed only with nosuchprog?**
   The `exec` family of system calls does not return on success. When `execlp("ls", ...)` succeeds, the Linux kernel completely overwrites the child process's address space (code segment, data segment, heap, and stack) with the executable image of `ls`. Execution resumes at the entry point of `ls`, so the original instructions following `execlp()` no longer exist in memory.
   The code immediately following `execlp()` (including `perror("exec")` and `exit(127)`) is reached **only** if `execlp()` encounters an error (such as `ENOENT` when the executable cannot be located in `$PATH`).

2. **Which number does the child print as ”my parent”, and whose PID is it?**
   The child prints `12725` as “my parent”, which matches the PID printed by the parent on the first line (`getpid()`). The `getppid()` system call directly queries the kernel process table for the parent process ID (`p_pptr->p_pid`).

3. **What status does the parent see with ls, and what status with nosuchprog?**
   With `ls`, the parent observes status `0`, which indicates successful completion without errors. With `nosuchprog`, the parent observes status `127`, which is the standard Unix exit status emitted by our child code via `exit(127)` when command execution fails.

1.3 Part 2. A zombie, on purpose

1.3.1 Constructing a Zombie Process: zombie.c

A **zombie process** (or *defunct process*) is a process that has completed execution via `exit()`, but whose entry remains in the operating system's process table because its parent process has not yet read its exit status via `wait()` or `waitpid()`.

We create `zombie.c` by taking `run.c` and inserting a 20-second sleep delay into the parent process immediately before calling `waitpid()`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* Part 2: A zombie, on purpose */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
```

```
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);
        perror("exec");
        exit(127);
    }

    int status;
    sleep(20);                                  /* the parent sleeps for 20s */
    waitpid(pid, &status, 0);                   /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

1.3.2 Process Table Observation

We compile the program, launch it in the background, and observe process states during the 20-second interval:

```
➜  os-lab7 gcc -Wall -Wextra -o zombie zombie.c
➜  os-lab7 ./zombie > /dev/null &
➜  os-lab7 sleep 2 ; ps -o pid,ppid,stat,cmd | grep -E "zombie|ls|PID"
    PID    PPID STAT CMD
  12783   12776 S<+  ./zombie
  12784   12783 Z<+  [ls] <defunct>

# After waiting 20 seconds:
➜  os-lab7 sleep 19 ; ps -o pid,ppid,stat,cmd | grep -E "zombie|ls|PID"
    PID    PPID STAT CMD
```

Observe:

1. **Which letter does ls have in the STAT column, and what does it mean?**
   `ls` has the letter `Z` (with modifier `<+`) in the `STAT` column, indicating state `TASK_ZOMBIE` (*defunct*).
   In this state, the child process has completely released its virtual memory address space, closed all open file descriptors, and freed physical RAM. However, its entry in the kernel's process descriptor table (`struct task_struct`) is preserved so that its PID, exit code (0), and execution accounting statistics remain available for the parent process to read.

2. **After 20 seconds, is the [ls] ¡defunct¿ line still there? Which line of zombie.c made it disappear?**
   No, after 20 seconds the `[ls] <defunct>` line is completely gone from the process table. The line that eliminated it was:

   ```
   waitpid(pid, &status, 0);
   ```

   Once the parent's `sleep(20)` finished, the parent invoked `waitpid()`. This system call collected the child's termination status code, acknowledging its demise. The kernel subsequently deallocated the child's task descriptor and released PID 12784 back to the operating system's PID pool.

1.4 Part 3. Redirection: ls -l ¿ out.txt

1.4.1 Descriptor Table Manipulation: redirect.c

In Unix systems, file descriptors are small non-negative integers that index into a per-process open file table maintained by the kernel. Standard streams are mapped by convention:

- **0**: Standard Input (`stdin`)
- **1**: Standard Output (`stdout`)
- **2**: Standard Error (`stderr`)

POSIX specifies that when a file is opened, the operating system **must allocate the lowest-numbered file descriptor currently available**. Shell redirection leverages this principle: the shell closes descriptor 1, calls `open()` on the target destination (which occupies slot 1), and then calls `exec()`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:
   ls -l > out.txt
*/
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                                   /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

1.4.2 Execution and Output Verification

```
➜  os-lab7 gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
➜  os-lab7 head -5 out.txt
total 72
-rw-r--r-- 1 ionut ionut     0 Oct 19 15:40 out.txt
-rwxr-xr-x 1 ionut ionut 16416 Oct 19 15:40 redirect
-rw-r--r-- 1 ionut ionut   702 Oct 19 15:40 redirect.c
-rwxr-xr-x 1 ionut ionut 16472 Oct 19 15:39 run
```

Observe:

1. **Which number did open give, and why exactly that one?**
   `open()` returned file descriptor `1`.
   Under POSIX rules, the kernel always assigns the lowest available integer index in the process's descriptor table. Because `close(1)` freed slot 1 (while slot 0 remained attached to `stdin`), the next call to `open()` was guaranteed to receive descriptor 1.

2. **ls was not changed at all. Why does its listing not appear on the screen?**
   The executable code of `ls` does not know or care whether descriptor 1 points to a terminal display, a file, a pipe, or a network socket. It simply emits its output using system call `write(1, buffer, count)`. Because descriptor 1 was rebound to `out.txt` prior to `execlp()`, all bytes written by `ls` to descriptor 1 were routed directly to the inode blocks of `out.txt` on disk.

3. **To redirect the errors instead (like 2¿ err.txt), which descriptor would you close before open?**
   To redirect standard error, you would execute `close(2)`. The subsequent `open("err.txt", ...)` would allocate descriptor **2**, causing all error diagnostics written to `stderr` to be directed into the file.

1.5 Part 4. A pipe: ls /etc — wc -l

1.5.1 Inter-Process Communication: pipeline.c

A pipe is a unidirectional kernel-buffered FIFO byte stream with two file descriptors:

- `p[0]`: The reading end.
- `p[1]`: The writing end.

To connect `ls /etc` to `wc -l`:

- Child 1 binds its standard output (1) to `p[1]` via `dup(p[1])`.
- Child 2 binds its standard input (0) to `p[0]` via `dup(p[0])`.
- The parent closes both pipe ends so that it does not hold open references.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:
   ls /etc | wc -l
*/
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
```

```
19          execlp("ls", "ls", "/etc", (char *)NULL);
20          perror("exec ls");
21          exit(127);
22      }
23
24      if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
25          close(0);
26          dup(p[0]);                              /* smallest free number: 0 */
27          close(p[0]);
28          close(p[1]);
29          execlp("wc", "wc", "-l", (char *)NULL);
30          perror("exec wc");
31          exit(127);
32      }
33
34      close(p[0]);                                /* the parent uses neither end */
35      close(p[1]);
36      wait(NULL);
37      wait(NULL);
38      return 0;
39  }
```

1.5.2 Comparison with Standard Shell Pipeline

```
➜  os-lab7 gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
215
➜  os-lab7 ls /etc | wc -l
215
```

Both executions produce the exact same count: **215 lines**.

1.5.3 Intentional Bug Experiment: The Hanging Reader

We test the consequences of failing to close unused pipe descriptors by commenting out line 35 (`close(p[1]);`) in the parent process:

```
# In pipeline.c, parent process:
#   close(p[0]);
#   /* close(p[1]); */
➜  os-lab7 gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
# Program hangs indefinitely until interrupted via Ctrl+C
```

Observe:

1. **Did ./pipeline and ls /etc — wc -l give the same number?**
   Yes, both commands returned 215, proving that our C implementation faithfully replicates the shell pipeline.

2. **After the change, which of the two children never ends, ls or wc?**
   `wc` never ends.
   The `ls` child executes, writes all directory entries into `p[1]`, closes its file descriptors upon termination, and exits immediately. However, `wc` remains blocked inside the `read()` system call waiting for additional input.

3. **Why, in one sentence: what is wc waiting for?**
   `wc` is waiting for an End-Of-File (EOF) indication on descriptor 0, which the kernel delivers only when **every** open file descriptor pointing to the write end of the pipe across the entire system has been closed; because the parent process never closed its inherited copy of `p[1]`, the pipe remains open indefinitely.

1.6 Optional Extension: Three-Stage Pipeline (pipeline3.c)

We extend the pipeline architecture to chain three distinct commands across two anonymous pipes:

$$\texttt{ls /usr/bin} \xrightarrow[\text{pipe 1}]{} \texttt{grep x} \xrightarrow[\text{pipe 2}]{} \texttt{wc -l}$$

1.6.1 Implementation: pipeline3.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* Three-stage pipeline:
   ls /usr/bin | grep x | wc -l
*/
int main(void)
{
    int p1[2], p2[2];

    if (pipe(p1) < 0) { perror("pipe 1"); return 1; }
    if (pipe(p2) < 0) { perror("pipe 2"); return 1; }

    /* Stage 1: ls /usr/bin -> p1[1] */
    if (fork() == 0) {
        close(1);
        dup(p1[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("ls", "ls", "/usr/bin", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    /* Stage 2: p1[0] -> grep x -> p2[1] */
    if (fork() == 0) {
        close(0);
        dup(p1[0]);
        close(1);
        dup(p2[1]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("grep", "grep", "x", (char *)NULL);
        perror("exec grep");
        exit(127);
    }

    /* Stage 3: p2[0] -> wc -l */
```

```
    if (fork() == 0) {
        close(0);
        dup(p2[0]);
        close(p1[0]); close(p1[1]);
        close(p2[0]); close(p2[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    /* Parent closes all 4 pipe ends and waits for all children */
    close(p1[0]); close(p1[1]);
    close(p2[0]); close(p2[1]);

    wait(NULL);
    wait(NULL);
    wait(NULL);

    return 0;
}
```

1.6.2 Verification

```
➜  os-lab7 gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
698
➜  os-lab7 ls /usr/bin | grep x | wc -l
698
```

Both our custom C program and the interactive shell report exactly **698 matching binaries**.

2 Conclusion

Through this laboratory work, I investigated the foundational process lifecycle, stream redirection, and inter-process communication mechanisms of the Linux operating system.

Implementing `run.c` demystified the separation of process instantiation from program execution: `fork()` clones the caller's virtual address space, while `exec()` replaces the memory image with a new binary, allowing the parent to track the child's exit status via `waitpid()`. The intentional creation of a zombie process in `zombie.c` demonstrated that terminated processes linger in state `Z` until reaped by their parent, illustrating how process tables prevent memory leaks while preserving diagnostic termination codes.

Developing `redirect.c` revealed the elegance of the Unix descriptor table: by leveraging the lowest-available-descriptor allocation rule, closing descriptor 1 enables transparent file redirection without modifying downstream application logic. Finally, authoring `pipeline.c` and `pipeline3.c` demonstrated how anonymous pipes synchronize independent processes through kernel FIFOs, highlighting that strict descriptor closure is vital to signaling EOF and preventing reader deadlocks. Together, these exercises demonstrated how simple primitive system calls compose into the powerful command-line environment of modern operating systems.